NumPy & Matplotlib

A quick reference of the most-used functions and their key parameters.
`import numpy as np`, `import matplotlib.pyplot as plt`.

NumPy — Array Creation

```
np.array([1,2,3], dtype=float) # from list/tuple
np.asarray(x)              # ndarray, no copy if possible
np.zeros(shape)            # zeros; shape=int or (r,c)
np.ones(shape)             # ones
np.full(shape, value)      # constant-filled
np.empty(shape)            # uninitialised (fast)
np.zeros_like(a)           # match shape+dtype of a
np.ones_like(a)            # (also np.full_like)
np.arange(start,stop,step)# range-like, [start,stop)
np.linspace(a,b,num=50)    # evenly spaced, endpoints incl.
np.logspace(a,b,num)       # 10**a .. 10**b
np.eye(N)                  # N x N identity
np.meshgrid(x,y)           # 2-D grids -> X, Y
```

NumPy — Shape & Attributes

```
a.shape         # dimensions (tuple)
a.ndim          # number of axes
a.size          # total elements
a.dtype         # float64,int64,complex128,...
a.T             # transpose (view)
a.astype(float)# cast to new dtype (copy)
a.reshape(r,c) # new shape (-1 infers one axis)
a.ravel()       # flatten to 1-D (view)
a.flatten()     # flatten to 1-D (copy)
np.expand_dims(a,axis)  # insert size-1 axis
np.squeeze(a)           # drop size-1 axes
np.concatenate([a,b],axis=0)
np.stack([a,b],axis=0)  # join on NEW axis
np.hstack([a,b]); np.vstack([a,b])
np.split(a,k,axis=0)    # k equal parts
np.tile(a,reps)         # repeat whole array
np.repeat(a,n)          # repeat each element
np.roll(a,shift,axis)   # circular shift
np.pad(a,width,mode='constant')
```

NumPy — Indexing & Selection

```
a[i,j]          # single element
a[1:5:2]        # slice start:stop:step
a[a>0]          # boolean mask -> 1-D
a[[0,2,4]]      # fancy (integer) index
np.where(cond,x,y) # pick x where cond else y
np.where(cond)     # -> indices where True
np.nonzero(a)      # indices of nonzeros
np.clip(a,lo,hi)   # clamp to [lo,hi]
np.take(a,idx)     # gather by index
```

NumPy — Elementwise Math

```
+ - * / **        # elementwise (broadcasted)
np.add,subtract,multiply,divide,power
np.sqrt(x); np.square(x); np.cbrt(x)
np.exp(x); np.log(x); np.log10; np.log2
np.expm1; np.log1p        # accurate near 0
np.sin,cos,tan,arctan2(y,x)
np.sinh,cosh,tanh
np.abs(x); np.sign(x)
np.floor,ceil,trunc; np.round(x,decimals)
np.mod(x,y)  (x % y); np.remainder
np.maximum(a,b); np.minimum(a,b) # elementwise
np.deg2rad; np.rad2deg
```

NumPy — Reductions

Most accept `axis=None` *and* `keepdims=False`.

```
np.sum(a,axis=None)   # total or along axis
np.prod(a,axis)
np.mean; np.std; np.var; np.median
np.min; np.max; np.ptp   # ptp = max - min
np.argmin; np.argmax     # index of extreme
np.cumsum; np.cumprod    # running totals
np.percentile(a,q); np.quantile(a,q)
np.all(cond); np.any(cond)
np.count_nonzero(a)
```

NumPy — Calculus & Signals

```
np.trapezoid(y,x=None,dx=1.0) # trapezoid integral
                              # NumPy<2.0: np.trapz
np.gradient(y,x)   # numerical derivative dy/dx
np.diff(a,n=1)     # differences a[i+1]-a[i]
np.cumsum(y)*dx    # running (Riemann) integral
np.interp(xq,xp,fp)# 1-D linear interpolation
np.convolve(a,v,mode='full') # 'same','valid'
np.correlate(a,v,mode='valid')
```

NumPy — Linear Algebra

```
a @ b             # matrix product (np.matmul)
np.dot(a,b)       # dot / mat-vec
np.inner; np.outer; np.cross
np.linalg.solve(A,b)  # solve A x = b (preferred)
np.linalg.inv(A)      # inverse (avoid if solvable)
np.linalg.det(A)      # determinant
np.linalg.norm(x,ord=2)   # vector/matrix norm
np.linalg.eig(A)      # eigenvalues, eigenvectors
np.linalg.svd(A)      # singular value decomp.
np.linalg.qr(A); np.linalg.cholesky(A)
```

NumPy — Logic, Sorting, Complex

```
np.isclose(a,b,rtol,atol) # elementwise ~equal
np.allclose(a,b)          # single bool
np.array_equal(a,b)
np.logical_and/or/not/xor
np.isnan; np.isinf; np.isfinite
np.unique(a,return_counts=True)
np.sort(a,axis=-1); np.argsort(a)
# complex:
np.conj(z); np.angle(z,deg=False); np.abs(z)
z.real; z.imag; 1j   # imaginary unit
```

NumPy — Random & Constants

```
rng = np.random.default_rng(seed)  # new API
rng.random(size)          # uniform [0,1)
rng.normal(loc,scale,size)
rng.uniform(low,high,size)
rng.integers(low,high,size)
rng.choice(a,size,replace=True)
rng.shuffle(a)            # in-place
# constants:
np.pi; np.e; np.inf; np.nan; np.newaxis
```

Matplotlib — Figure & Axes

```
fig, ax = plt.subplots(figsize=(w,h)) # 1 Axes (best)
fig, axs = plt.subplots(nrows,ncols,
     figsize=(w,h), sharex=True, sharey=False)
plt.figure(figsize=(8,4), dpi=100)
ax = fig.add_subplot(1,1,1)
fig.tight_layout()      # remove overlaps
fig.suptitle("title")   # figure-level title
plt.show()              # open a window
fig.savefig("out.png", dpi=150,
            bbox_inches="tight")
plt.close(fig)
# headless (scripts / servers):
import matplotlib; matplotlib.use("Agg")
```

Matplotlib — Lines & Scatter

```
ax.plot(x,y, color="C0", linewidth=1.5,
   linestyle="--", marker="o", markersize=4,
   label="x(t)", alpha=1.0)
ax.plot(x,y,"r--o")         # fmt: colour,style,marker
ax.scatter(x,y, s=20, c="red", alpha=.6)
ax.stem(n,x)                # discrete-time signals
ax.step(x,y, where="mid")  # staircase
ax.fill_between(x,y1,y2, alpha=.2)
ax.errorbar(x,y, yerr=e, fmt="o")
ax.loglog(x,y); ax.semilogx(x,y); ax.semilogy(x,y)
```

Matplotlib — Bars, Hist, Images

```
ax.bar(x,h, width=0.8); ax.barh(y,w)
ax.hist(data, bins=30, density=True)
im = ax.imshow(M, cmap="viridis",
   aspect="auto", origin="lower",
   extent=[x0,x1,y0,y1])
ax.pcolormesh(X,Y,Z, shading="auto")
ax.contour(X,Y,Z, levels); ax.contourf(...)
fig.colorbar(im, ax=ax)
```

Matplotlib — Labels, Limits, Ticks

```
ax.set_xlabel("t"); ax.set_ylabel("x(t)")
ax.set_title("...")
ax.set_xlim(a,b);  ax.set_ylim(a,b)
ax.set_xticks([...]); ax.set_xticklabels([...])
ax.tick_params(labelsize=8)
ax.grid(True, alpha=.3, linestyle=":")
ax.axhline(0, color="k", lw=.8)  # h ref line
ax.axvline(x0, color="k")        # v ref line
ax.axvspan(a,b, color="orange", alpha=.15) # band
ax.set_aspect("equal"); ax.invert_yaxis()
```

Matplotlib — Legend, Text, Annotate

```
ax.legend(loc="best", fontsize=8,
          ncol=2, frameon=True)
ax.text(x,y, "label", fontsize=9, ha="center")
ax.annotate("peak", xy=(x,y), xytext=(x+1,y+1),
   arrowprops=dict(arrowstyle="->"))
axt = ax.twinx()   # shared x, right-hand y
for a in axs.flat: a.grid(True)  # loop a grid
```

Matplotlib — Styling Reference

```
# colours : "C0".."C9" cycle; "red","k","0.6"(grey)
# cmaps   : viridis, plasma, coolwarm, gray, jet
# lines   : "-"  "--"  "-."  ":"
# markers : o  .  x  s  ^  *  +
# align   : ha=left/center/right, va=top/bottom
plt.style.use("seaborn-v0_8")  # preset look
plt.rcParams["font.size"] = 11 # global default
```